L. Tulasi
192311250

Assignment - 2

(1) Regular Expression Design for input validation

1. Binary strings that end with 01

Regular Expression:

$(0+1)^* 01$

2. Binary strings containing the substring 11 or 00

Regular Expression:

$(0+1)^* (11+00) (0+1)^*$

3. Substring over {a,b} containing at least one occurence of ab

Regular Expression:

$(a+b)^* ab (a+b)^*$

4. Strings over {0,1} having an Even number of zeros.

Regular Expression:

$1^* (01^* 01^*)^*$

5. strings over {a,b} that start with a and end with b.

Regular Expression:

$a(a+b)^* b$

(2) Regular Expression Analysis and language Repres

Given regular expression:

$(0+1)^* 11 (0+1)^*$

1. Language Representation:

It represents all binary strings containing the substring 11 at least once

2. Alphabet:

$\Sigma = \{0, 1\}$

3. Substring recognized

11

4. Five accepted strings.

1. 11
2. 011
3. 110
4. 111
5. 1011

5. Three rejected strings

1. 0
2. 1

Explanation of operators

Kleene star (*):

* Represents zero or more repetitions.

* (0+1)* allows any Combination of 0's & 1's

Union (+):

* Represents a choice between two Symbols.

* 0+1 means Either 0 or 1.

Conclusion:-

The regular Expression accepts all binary strings Containing at least one occurrence of 11. and rejects strings that do not contain 11.

3. Context-Free Grammar Construction

1. Arithmetic Expressions involving Identifiers & Addition(+)

Grammar Components:

* Start Symbols : S

* Non-terminals : S, E

* Terminals : id, +

Production rules:-

S → E

E → E+E

E → id

Ex:- id, id+id, id+id+id.

2. Balanced parentheses:

Grammar Components:

⇒ Start Symbol : S

⇒ Non-terminal : S

⇒ Terminals : ( )

Production rules:-

$S \rightarrow (S)$

$S \rightarrow SS$

$S \rightarrow E$

3. Arithmetic Expressions involving Identifiers and additions (+)

Grammar Components:-

* Start Symbol : E
* Non-terminal : E
* Terminals : id, +

Production rules:-

$E \rightarrow E+E$

$E \rightarrow id$

4. Balanced paranthesis :-

Grammar Components:

* Start Symbol : S
* Non-terminal : S
* Terminals : (,)

Production rules:-

$S \rightarrow (S)$

$S \rightarrow SS$

$S \rightarrow \varepsilon$

5. Binary strings Containing an Equal number of zero's and one's:

Grammar Components:

* Start Symbol : S
* Non-terminal : S
* Terminals : 0, 1

Production rules:-

$S \rightarrow 0S1S$

$S \rightarrow 1S0S$

$S \rightarrow \varepsilon$

Ex:- ε, 01, 10, 0011, 0101

Result:-

The five CFG's are Constructed with their start symbols, terminals, non-terminals are and production rules.

4) Derivation and parse Tree construction

Given Context-Free Grammar (CFG):

$$S \rightarrow asb \mid \varepsilon$$

1. Components of Grammar.

* Start Symbol: S
* Non-terminals: S
* Terminals: a, b
* Production rules:
  * $S \rightarrow aSb$
  * $S \rightarrow \varepsilon$

2. Generate strings for $n = 0, 1, 2, 3$

n	Derivation	Generated String
0	$S \rightarrow \varepsilon$	ε
1	$S \rightarrow aSb \rightarrow ab$	ab
2	$S \rightarrow aSb \rightarrow aaSbb \rightarrow aabb$	aabb
3	$S \rightarrow aSb \rightarrow aaSbb \rightarrow aaaSbbb \rightarrow aaabbb$	aaabbb

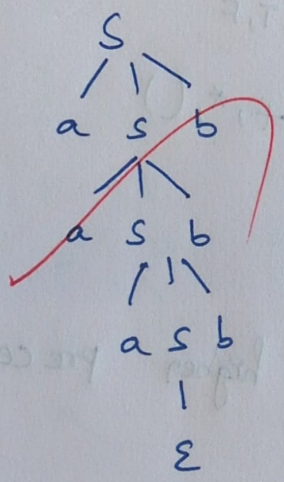
left most derivation: of aaabbb :-

S
=> asb
=> aasbb
=> aaaSbbb
=> aaabbb

4. Right most derivation of aaabbb :-

S
=> aSb
=> aaSbb
-> aaaSbbb
=> aaabbb

5. Parse Tree for aaabbb



Result:-

The string aaabbb is successfully generated Using the given assumed CFG. Both leftmost and derivations produce the same string.

Grammar Design for Arithmetic Expressions.

1. Context Free Grammar.

Production rules:-

$E \rightarrow E+T$

$E \rightarrow T$

$T \rightarrow T * F$

$T \rightarrow F$

$F \rightarrow (E)$

$F \rightarrow id$

2. Grammar Components :-

* Start Symbol : E
* Non-terminals : E, T, F
* Terminals : id, +, *, ( )

3. Operator precedence :-

* Multiplication (*) has higher precedence than addition (+).
* Addition (+) has lower precedence.
* parentheses are used to group Expressions

Derivation of a+b*c :-

E

=> E+T

=> T+T

=> F+T

-> a+T

-> a+T* F

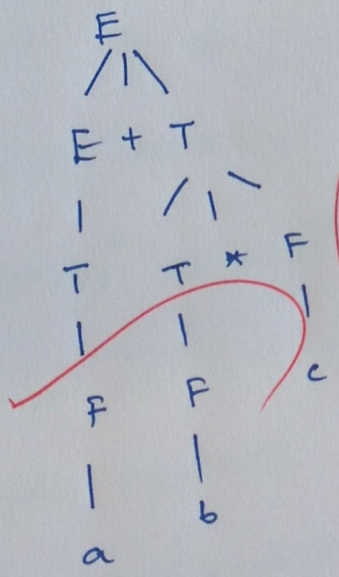
-> a +F* F

-> a+b *F

-> a+b*c

Final string: a+b*c

Parse tree:-



Result:-

The CFG Successfully validates arithmetic Expressions Containing identifiers, addition, multiplication and parentheses.

- It also maintains the correct operator Precedence, where multiplication has higher Precedence than addition.